

Teste de Software

Refatoração de Testes e Detecção de Test Smells

Relatório do trabalho prático

Luísa Oliveira Jardim

Repositório: github.com/luisajardim/test-smelly-PUC-

1. Análise de Smells

O arquivo original tem 4 testes e todos passam. Só que, lendo com calma, deu pra ver que vários deles têm problemas sérios. Alguns são difíceis de entender, outros quebram por qualquer motivo e um deles nem consegue pegar o bug que deveria. Escolhi os três smells que achei mais graves.

1.1 Lógica Condicional no Teste

Onde aparece: no teste *“deve desativar usuários se eles não forem administradores”*. Ele usa um `for` para passar pelos usuários e um `if/else` para decidir qual `expect` vai rodar.

Por que é um mau cheiro: um teste deveria seguir sempre o mesmo caminho. Quando tem laço e condicional, o teste vira um código que também pode ter bug, e pra entender o que ele verifica eu preciso simular a execução na cabeça. Ele também testa duas regras diferentes de uma vez: a do usuário comum e a do administrador.

Risco: se alguma das regras quebrar, a mensagem de erro não deixa claro qual foi. E se a lista estiver vazia ou alguém mudar o `if`, os `expect` podem nem rodar. Nesse caso o teste passa sem ter verificado nada.

1.2 Teste de Exceção que Passa Sem Verificar Nada

Onde aparece: no teste *“deve falhar ao criar usuário menor de idade”*. O `expect` fica dentro do `catch`.

Por que é um mau cheiro: se o código não lançar a exceção, o `catch` nunca é executado e o teste termina sem nenhuma verificação. Pro Jest, isso conta como sucesso. Ou seja, o teste confere a mensagem do erro, mas não confere se o erro realmente aconteceu.

Risco: é um falso positivo. Pra provar isso, apaguei por um momento a validação de idade mínima no código de produção. O arquivo original **continuou passando** e o meu arquivo refatorado **falhou**, que é o certo. Com a suíte original, esse bug chegaria em produção sem ninguém perceber.

1.3 Teste Frágil

Onde aparece: no teste *“deve gerar um relatório de usuários formatado”*. Ele compara uma linha inteira do relatório, caractere por caractere, e ainda exige que o texto comece com um cabeçalho exato.

Por que é um mau cheiro: o teste está preso ao formato da string, que é um detalhe de implementação. O que importa de verdade é se o relatório mostra os usuários e seus status. O próprio código-fonte tem um comentário dizendo que o formato do relatório pode mudar no futuro.

Risco: qualquer mudança visual quebra o teste, mesmo que o sistema continue funcionando. Basta um espaço a mais ou trocar a ordem dos campos. Com o tempo, a equipe se acostuma com testes falhando à toa e passa a ignorar ou apagar esses testes.

Outros smells que encontrei

Eager Test: o primeiro teste cria e busca um usuário no mesmo teste, então verifica duas coisas de uma vez.

Teste desabilitado e vazio: o `test.skip` tem só um *TODO* e passa a impressão de que o caso sem usuários está coberto, quando não está.

2. Processo de Refatoração

Escolhi o teste de desativação de usuários porque ele sozinho gera 3 dos 4 erros do ESLint e mistura duas regras de negócio no mesmo fluxo.

Antes: userService.smelly.test.js

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois: userService.clean.test.js

```
describe('deactivateUser', () => {
  test('deve desativar um usuário comum e retornar true', () => {
    // Arrange
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);

    // Assert
    expect(resultado).toBe(true);
    expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
  });

  test('não deve desativar um usuário administrador e deve retornar false', () => {
    // Arrange
    const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

    // Act
    const resultado = userService.deactivateUser(usuarioAdmin.id);

    // Assert
    expect(resultado).toBe(false);
    expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
  });
});
```

O que eu fiz e por quê:

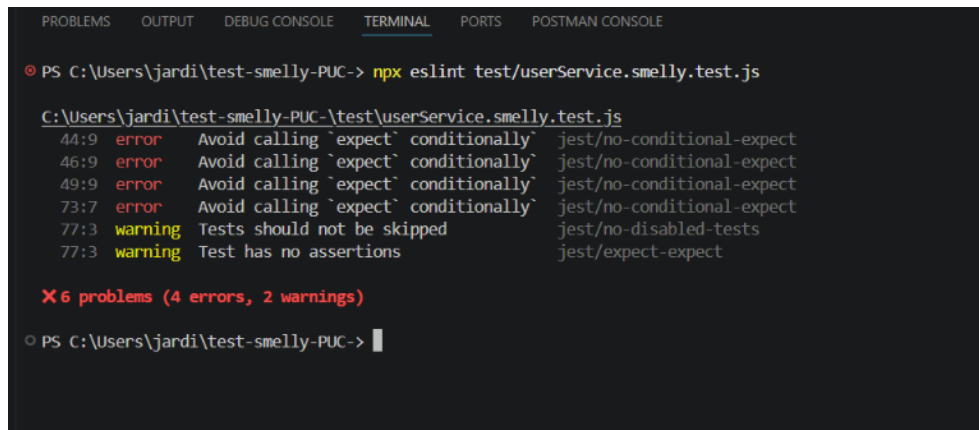
- **Um teste pra cada regra.** Separei em dois testes, um pro usuário comum e outro pro administrador. Com isso o for e o if/else deixaram de ser necessários, todos os expect sempre rodam e os erros de no-conditional-expect sumiram.
- **Padrão AAA.** Cada teste prepara só os dados que precisa, chama o método uma única vez e depois verifica o resultado. Deixei os comentários Arrange, Act e Assert pra ficar fácil de ver cada parte.
- **Nomes claros.** Quando um teste falha, o nome já diz qual regra quebrou. Também agrupei os testes por método usando describe.
- **Verificação mais completa.** No caso do administrador, agora também confiro se o status continua “ativo”. Antes o teste só olhava o retorno.

Nos outros testes segui a mesma ideia. Troquei o try/catch pelo toThrow do Jest, que falha se a exceção não for lançada. No teste do relatório, passei a verificar só se o nome e o status de cada usuário aparecem, sem depender do formato exato. O test.skip virou um teste de verdade pro relatório vazio. Também criei testes para casos que não existiam: campos obrigatórios faltando, busca por um id que não existe e desativação de um usuário que não existe. No fim, a suíte limpa ficou com 11 testes.

3. Relatório da Ferramenta

Configurei o ESLint com o plugin *eslint-plugin-jest* usando o arquivo *.eslintrc.json* do enunciado. Precisei instalar a versão 8 do ESLint, porque a partir da versão 9 ele não lê mais esse tipo de arquivo de configuração.

Esse foi o resultado da primeira execução no arquivo original:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\jardi\test-smelly-PUC-> npx eslint test/userService.smelly.test.js

C:\Users\jardi\test-smelly-PUC-\test\userService.smelly.test.js
44:9 error Avoid calling `expect` conditionally jest/no-conditional-expect
46:9 error Avoid calling `expect` conditionally jest/no-conditional-expect
49:9 error Avoid calling `expect` conditionally jest/no-conditional-expect
73:7 error Avoid calling `expect` conditionally jest/no-conditional-expect
77:3 warning Tests should not be skipped jest/no-disabled-tests
77:3 warning Test has no assertions jest/expect-expect

X 6 problems (4 errors, 2 warnings)

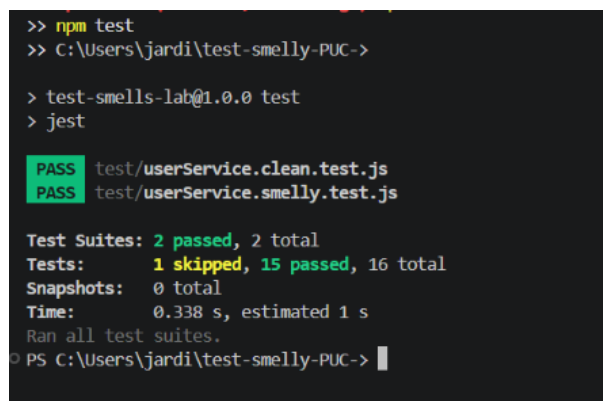
PS C:\Users\jardi\test-smelly-PUC->
```

Figura 1: primeira execução do ESLint no arquivo original, com 4 erros e 2 avisos.

O ESLint achou sozinho, em poucos segundos, dois dos smells que eu já tinha encontrado lendo o código. A regra no-conditional-expect apontou os expect dentro do if/else nas linhas 44, 46 e 49 e dentro do catch na linha 73. Já as regras no-disabled-tests e expect-expect apontaram o test.skip vazio na linha 77.

Mas ele não percebeu o Teste Frágil nem o Eager Test. Esses dois dependem de entender a intenção do teste, e isso a análise estática não consegue fazer. Então a ferramenta ajuda muito, mas não substitui alguém lendo o código.

Depois da refatoração, rodei o ESLint no arquivo novo e ele não mostrou nenhum erro nem aviso. Os testes dos dois arquivos continuam passando:



```
>> npm test
>> C:\Users\jardi\test-smelly-PUC->

> test-smells-lab@1.0.0 test
> jest

PASS test/userService.clean.test.js
PASS test/userService.smelly.test.js

Test Suites: 2 passed, 2 total
Tests: 1 skipped, 15 passed, 16 total
Snapshots: 0 total
Time: 0.338 s, estimated 1 s
Ran all test suites.
PS C:\Users\jardi\test-smelly-PUC->
```

Figura 2: npm test com as duas suítes passando. O teste pulado é o test.skip do arquivo original.

4. Conclusão

O que mais me marcou nesse trabalho foi perceber que um teste passar não quer dizer que ele é bom. A suíte original passava, mas um dos testes deixaria passar um bug importante e outro quebraria com qualquer mudança de formatação. Testes limpos, que verificam uma coisa por vez, seguem o AAA e têm nomes claros, funcionam quase como uma documentação do sistema. Quando falham, dá pra saber na hora o que quebrou. Quando passam, dá pra mexer no código com confiança.

O ESLint ajuda a manter essa qualidade ao longo do tempo, porque verifica as boas práticas sozinho toda vez que roda e pode ser colocado no CI pra barrar testes ruins antes de entrarem no projeto. Como alguns smells só uma pessoa percebe, o ideal é usar os dois juntos. A ferramenta cuida do que é repetitivo e a gente foca no que precisa de análise.